

任务 1：知识图谱本体推理

本实践任务要求每位同学独立完成提交

学习 Neo4j 的 neosemantics 插件：<https://neo4j.com/labs/neosemantics/>

(1) 为 Neo4j 安装 neosemantics 插件

<https://github.com/neo4j-labs/neosemantics/releases>

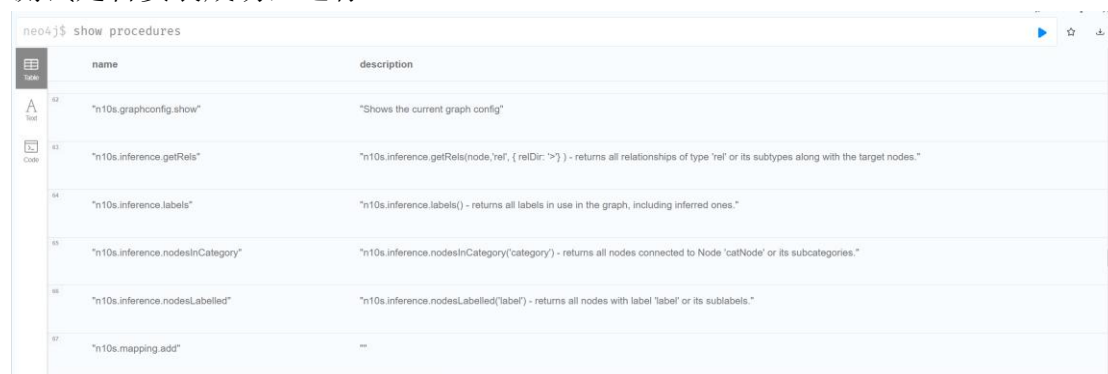
选择 5.26.0 版本

将下载的 jar 包放到 Neo4j 安装目录的 plugin 文件夹下，在 Neo4j 配置文件（conf/neo4j.conf）中添加插件，

```
359 #*****
360 # Other Neo4j system properties
361 #*****
362 dbms.unmanaged_extension_classes=n10s.endpoint=/rdf
```

重启 Neo4j

测试是否安装成功，运行 SHOW PROCEDURES



	name	description
359	"n10s.graphconfig.show"	"Shows the current graph config"
360	"n10s.inference.getRels"	"n10s.inference.getRels(node,'rel',{ relDir: '>' }) - returns all relationships of type 'rel' or its subtypes along with the target nodes."
361	"n10s.inference.labels"	"n10s.inference.labels() - returns all labels in use in the graph, including inferred ones."
362	"n10s.inference.nodesInCategory"	"n10s.inference.nodesInCategory('category') - returns all nodes connected to Node 'catNode' or its subcategories."
363	"n10s.inference.nodesLabelled"	"n10s.inference.nodesLabelled('label') - returns all nodes with label 'label' or its sublabels."
364	"n10s.mapping.add"	"--"

初始化插件，运行

```
CALL n10s.graphconfig.init({
  handleVocabUris: 'MAP'
})
```

建立约束，运行

```
CREATE CONSTRAINT n10s_unique_uri FOR (r:Resource)
  REQUIRE r.uri IS UNIQUE;
```



	name	description
365	"n10s_unique_uri"	"Added 1 constraint, completed after 237 ms."

插入 RDF 数据

```
CALL n10s.rdf.import.inline(
```

```

'
@prefix neo4voc: <http://neo4j.org/vocab/sw#> .
@prefix neo4ind: <http://neo4j.org/ind#> .

neo4ind:nsmntx3502 neo4voc:name "NSMNTX" ;
    a neo4voc:Neo4jPlugin ;
    neo4voc:version "3.5.0.2" ;
    neo4voc:releaseDate "03-06-2019" ;
    neo4voc:runsOn neo4ind:neo4j355 .

neo4ind:apoc3502 neo4voc:name "APOC" ;
    a neo4voc:Neo4jPlugin ;
    neo4voc:version "3.5.0.4" ;
    neo4voc:releaseDate "05-31-2019" ;
    neo4voc:runsOn neo4ind:neo4j355 .

neo4ind:graphql3502 neo4voc:name "Neo4j-GraphQL" ;
    a neo4voc:Neo4jPlugin ;
    neo4voc:version "3.5.0.3" ;
    neo4voc:releaseDate "05-05-2019" ;
    neo4voc:runsOn neo4ind:neo4j355 .

neo4ind:neo4j355 neo4voc:name "neo4j" ;
    a neo4voc:GraphPlatform , neo4voc:AwesomePlatform ;
    neo4voc:version "3.5.5" .

',
'Turtle'
)

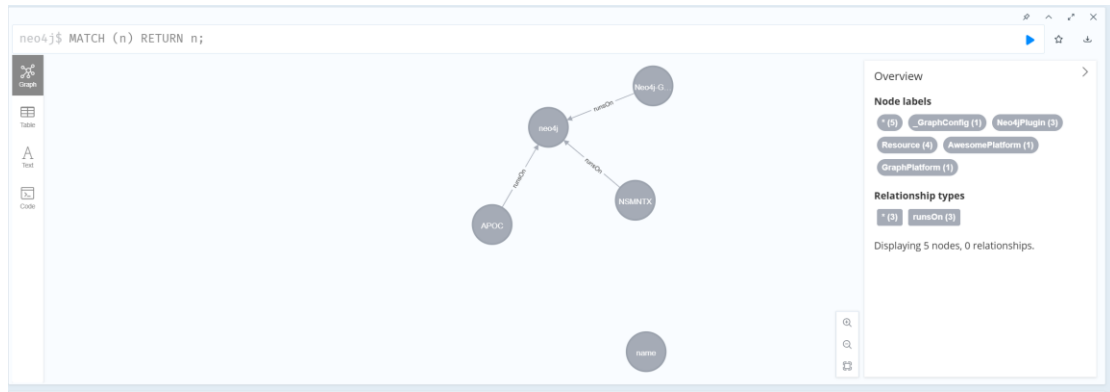
```



The screenshot shows a Neo4j Cypher query interface. The query executed is: `neo4j$ CALL n10s.rdf.import.inline( ' @prefix neo4voc: <http://neo4j.org/vocab/sw#> . @prefix neo4ind: <http://neo4j.org/ind#> . neo4ind:n...`. The result is displayed in a table with the following columns: `terminationStatus`, `triplesLoaded`, `triplesParsed`, `namespaces`, `extraInfo`, and `callParams`. The first row shows a successful import with `terminationStatus` as "OK", `triplesLoaded` as 19, and `triplesParsed` as 19. The `callParams` column shows a JSON object: `{ "singleTx": true }`. At the bottom, a status message reads: "Started streaming 1 records after 11 ms and completed after 311 ms."

	terminationStatus	triplesLoaded	triplesParsed	namespaces	extraInfo	callParams
1	"OK"	19	19	null	""	{ "singleTx": true }

查询 RDF 数据
MATCH (n) RETURN n;



清空数据库

`MATCH (n) DETACH DELETE n`

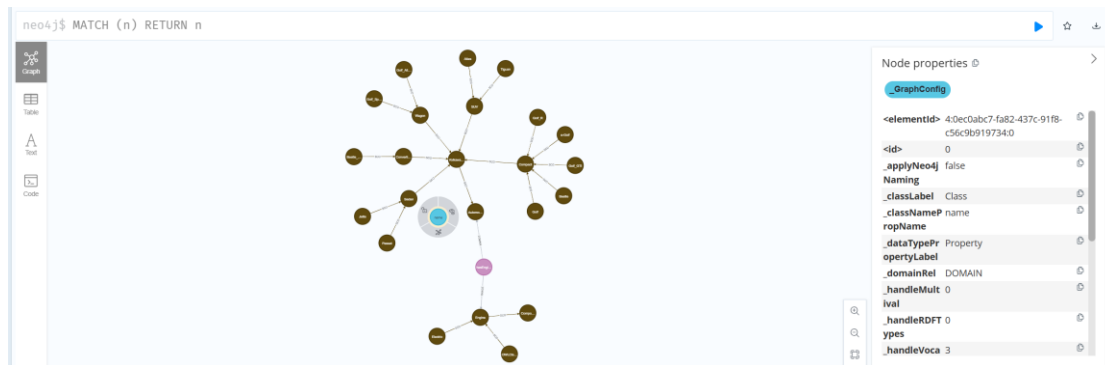
(2) 导入本体数据

```
CALL n10s.graphconfig.init({
  handleVocabUris: 'MAP'
})
```

`CALL n10s.onto.import.fetch`
 ("https://github.com/neo4j-labs/neosemantics/raw/3.5/docs/rdf/vw.owl","Turtle");
 如上述导入结果未成功（导入 0 个 triples），原因是网站访问问题。将该文件下载本地路径，如在 Windows 系统下为（路径根据自己本机上的位置修改）：
`CALL n10s.onto.import.fetch ("file:///C:\\tools\\vw.owl","Turtle");`

terminationStatus	triplesLoaded	triplesParsed	namespaces	extraInfo	callParams
"OK"	75	168	null	"	{ "singleTx": false }

`MATCH (n) RETURN n`



清空数据库

MATCH (n) DETACH DELETE n

(3) **RDF 数据类型推理**

`rdfs:subClassOf`

初始化插件:

```
CALL n10s.graphconfig.init({
  handleVocabUris: 'MAP'
})
```

a) **建立 RDF 层次类型数据**

运行下列代码:

```
CREATE (c:LCSHTopic { authoritativeLabel: "Crystallography", dbLabel: "Crystallography",
identifier: "sh 85034498" })

CREATE (po:LCSHTopic { authoritativeLabel: "Physical optics", dbLabel: "PhysicalOptics",
identifier: "sh 85095187" })

CREATE (s:LCSHTopic { authoritativeLabel: "Solids", dbLabel: "Solids", identifier: "sh
85124647" })

CREATE (c)-[:NARROWER_THAN]-(:LCSHTopic { authoritativeLabel: "Crystal optics", dbLabel:
"CrystalOptics", identifier: "sh 85034488" })-[:NARROWER_THAN]->(po)

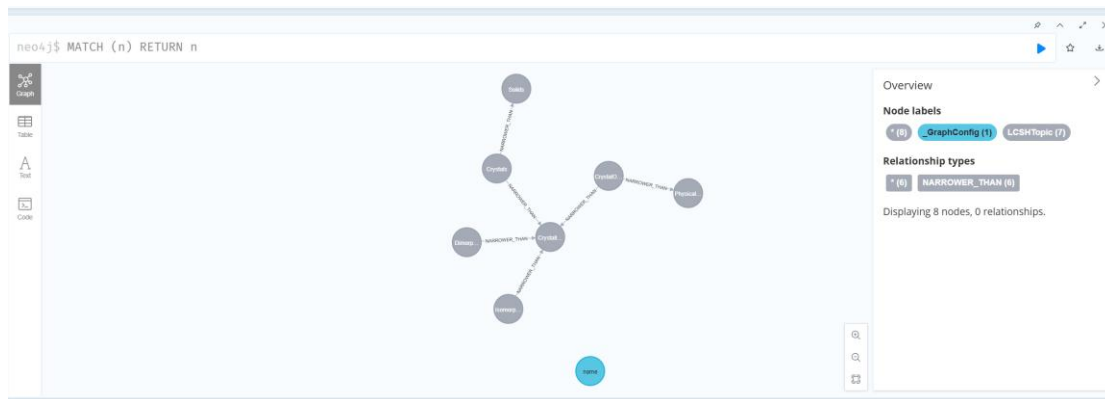
CREATE (c)-[:NARROWER_THAN]-(:LCSHTopic { authoritativeLabel: "Crystals", dbLabel:
"Crystals", identifier: "sh 85034503" })-[:NARROWER_THAN]->(s)

CREATE (c)-[:NARROWER_THAN]-(:LCSHTopic { authoritativeLabel: "Dimorphism
(Crystallography)", dbLabel: "DimorphismCrystallography", identifier: "sh 2007001101" })

CREATE (c)-[:NARROWER_THAN]-(:LCSHTopic { authoritativeLabel: "Isomorphism
(Crystallography)", dbLabel: "IsomorphismCrystallography", identifier: "sh 85068653" })
```



MATCH (n) RETURN n



b) 输入用于推理的数据

运行下列代码：

```
MATCH (co:LCSHTopic { authoritativeLabel: "Crystal optics"})
MATCH (is:LCSHTopic { authoritativeLabel: "Isomorphism
(Crystallography)"})
MATCH (cr:LCSHTopic { authoritativeLabel: "Crystals"})

CREATE (:Work { title: "Crystals and light", identifier: "2167673"})-
[:HAS_SUBJECT]->(co)
CREATE (:Work { title: "Optical crystallography", identifier:
"11916857"})-[:HAS_SUBJECT]->(co)

CREATE (:Work { title: "Isomorphism in minerals", identifier:
"8096161"})-[:HAS_SUBJECT]->(is)

CREATE (:Work { title: "Crystals and life", identifier: "12873809"})-
[:HAS_SUBJECT]->(cr)
CREATE (:Work { title: "Highlights in applied mineralogy", identifier:
"20234576"})-[:HAS_SUBJECT]->(cr);
```



MATCH (n) RETURN n


```
:param inferenceParams => ({ inCatRel: "HAS_SUBJECT", subCatRel: "NARROWER_THAN" });
```



c) 进行查询

```
MATCH (phyOpt:LCSHTopic { authoritativeLabel: "Physical optics"})
MATCH (:User { userId : "JB2020" })-[:INTERESTED_IN]->(b:Work)
WHERE n10s.inference.inCategory(b,phyOpt,$inferenceParams)
RETURN b
```



清空数据库

```
MATCH (n) DETACH DELETE n
```

(5) 关系层次推理

```
rdfs:subPropertyOf
```

初始化插件:

```
CALL n10s.graphconfig.init({
  handleVocabUris: 'MAP'
})
```

a) 建立数据

导入本体

```
CALL n10s.onto.import.inline('
@prefix owl: <http://www.w3.org/2002/07/owl#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#> .
```

```

@prefix neovoc: <neo4j://vocabulary#> .
@prefix neoind: <neo4j://individuals#> .

neovoc:ACTED_IN a owl:ObjectProperty;
  rdfs:label "ACTED_IN";
  rdfs:subPropertyOf neovoc:WORKED_IN .

neovoc:PRODUCED a owl:ObjectProperty;
  rdfs:label "PRODUCED";
  rdfs:subPropertyOf neovoc:WORKED_IN .

neovoc:WROTE a owl:ObjectProperty;
  rdfs:label "WROTE";
  rdfs:subPropertyOf neovoc:WORKED_IN .

neovoc:DIRECTED a owl:ObjectProperty;
  rdfs:label "DIRECTED";
  rdfs:subPropertyOf neovoc:WORKED_IN .

neovoc:WORKED_IN a owl:ObjectProperty;
  rdfs:label "WORKED_IN" .

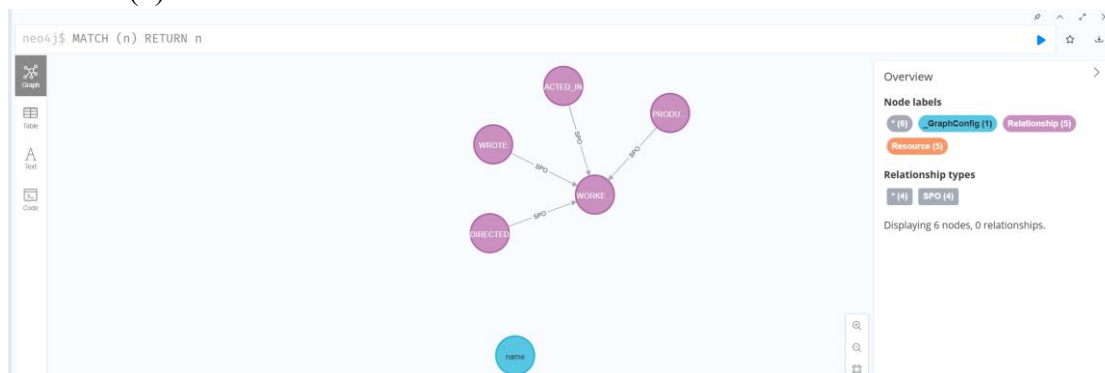
', 'Turtle');

```

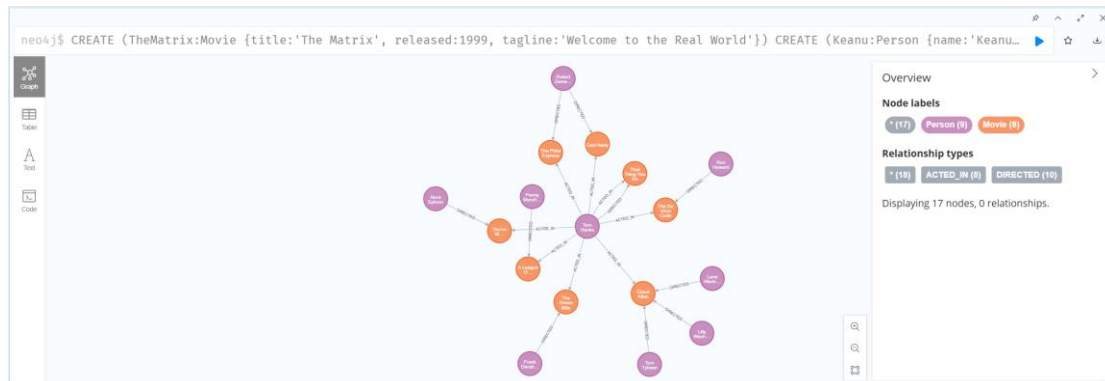
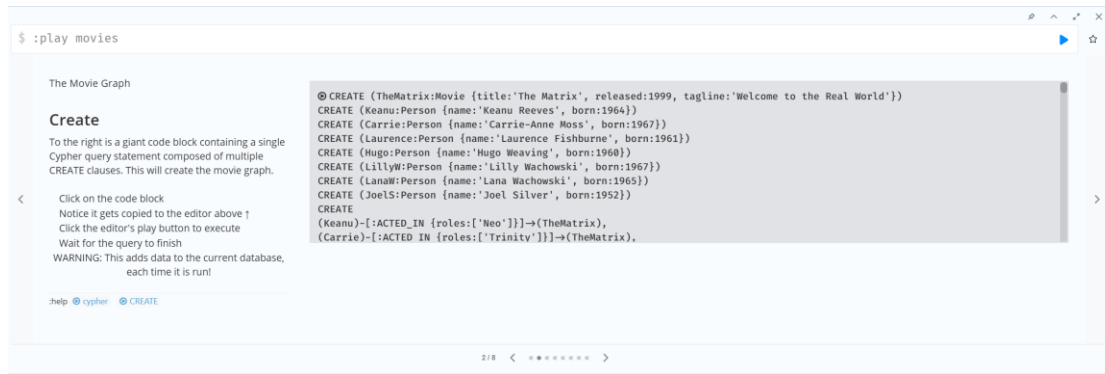


	terminationStatus	triplesLoaded	triplesParsed	namespaces	extraInfo	callParams
1	"OK"	14	14	null	--	{ "singleTx": true }

MATCH (n) RETURN n



加载:movie 示例数据
:play movies



b) 运用推理关系层次查询

```

match (thematrix:Movie {title: "The Matrix"})
call n10s.inference.getRels(thematrix,"WORKED_IN", { subRelRel: "SPO" })
yield rel, node
return type(rel) as relType, node

```

neo4j\$ match (thematrix:Movie {title: "The Matrix"}) call n10s.inference.getRels(thematrix,"WORKED_IN", { subRelRel: "SPO" }) yield rel, n...

relType	node
"DIRECTED"	(:Person {name: "Lana Wachowski", born: 1965})
"DIRECTED"	(:Person {name: "Lilly Wachowski", born: 1967})
"ACTED_IN"	(:Person {name: "Hugo Weaving", born: 1960})
"ACTED_IN"	(:Person {name: "Laurence Fishburne", born: 1961})
"ACTED_IN"	(:Person {name: "Carrie-Anne Moss", born: 1967})
"ACTED_IN"	(:Person {name: "Keanu Reeves", born: 1964})
"ACTED_IN"	(:Person {name: "Emil Eifrem", born: 1978})
"PRODUCED"	(:Person {name: "Joel Silver", born: 1952})

修改本体 WORKED_IN 关系不包括 PRODUCED

```

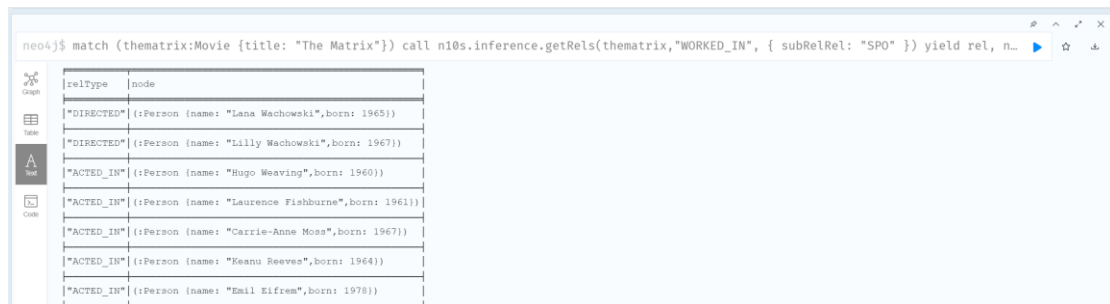
MATCH (:Relationship {name: "PRODUCED"}) -[r:SPO]-> (:Relationship
{name: "WORKED_IN"})
DELETE r

```



再次执行同样的查询

```
match (thematrix:Movie {title: "The Matrix"})
call n10s.inference.getRels(thematrix,"WORKED_IN", { subRelRel: "SPO" })
yield rel, node
return type(rel) as relType, node
```

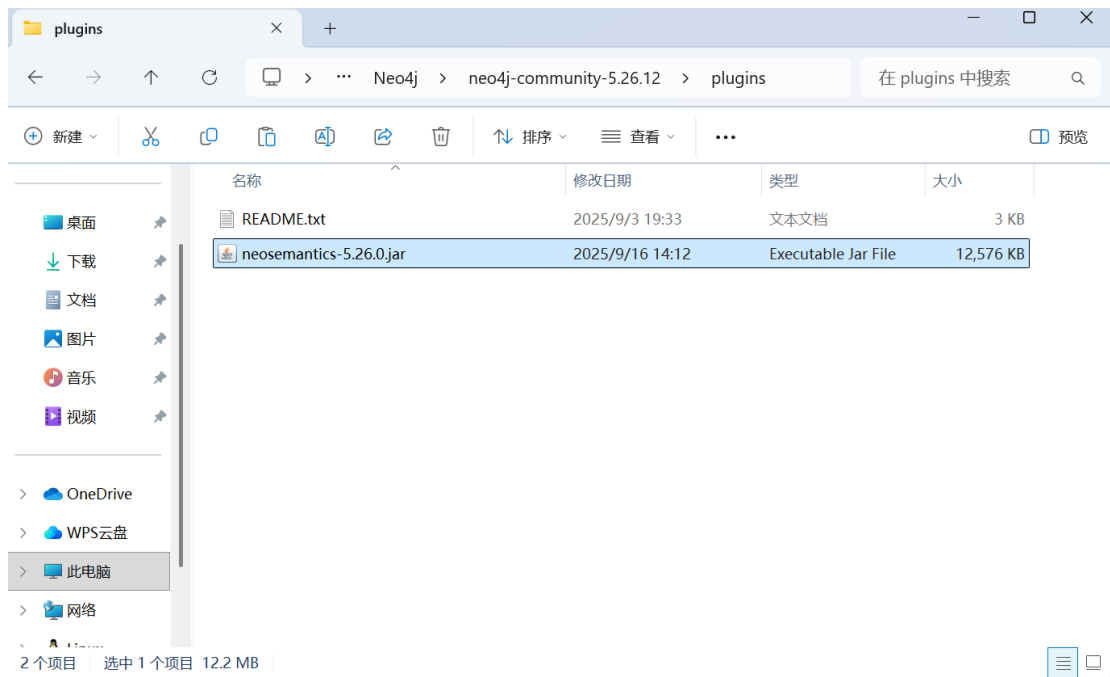


【要求】

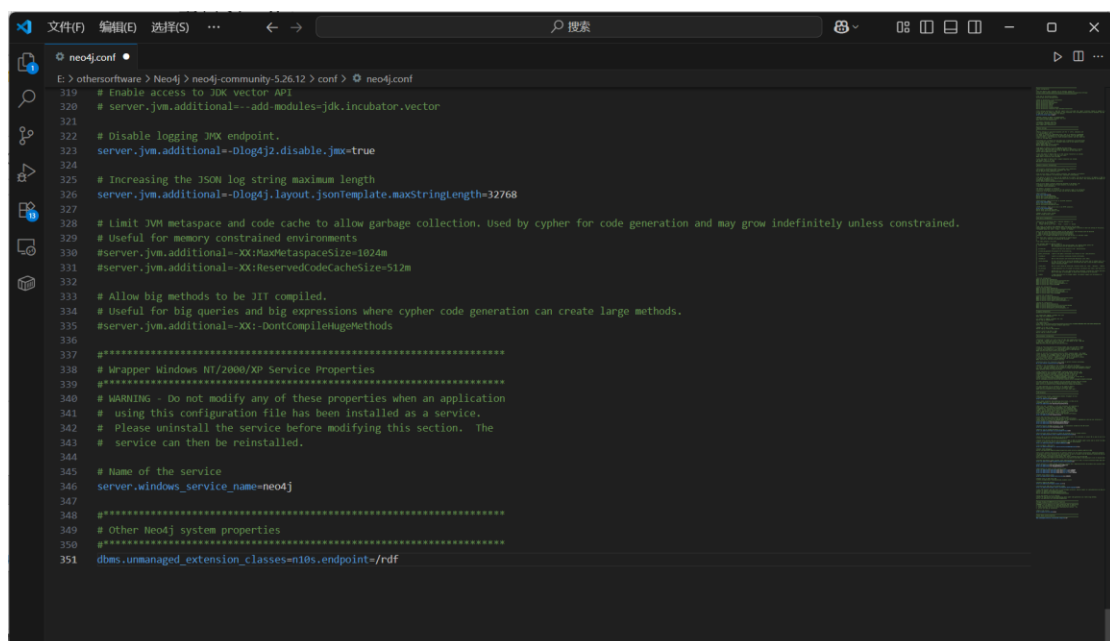
【每个小步骤（英文小括号如“a）”完成后的截图粘贴到下面】

(1) 为 Neo4j 安装 neosemantics 插件

将下载的 jar 包放到 Neo4j 安装目录的 plugin 文件夹下



然后在 Neo4j 配置文件（conf/neo4j.conf）中添加插件



重启 Neo4j，运行 SHOW PROCEDURES

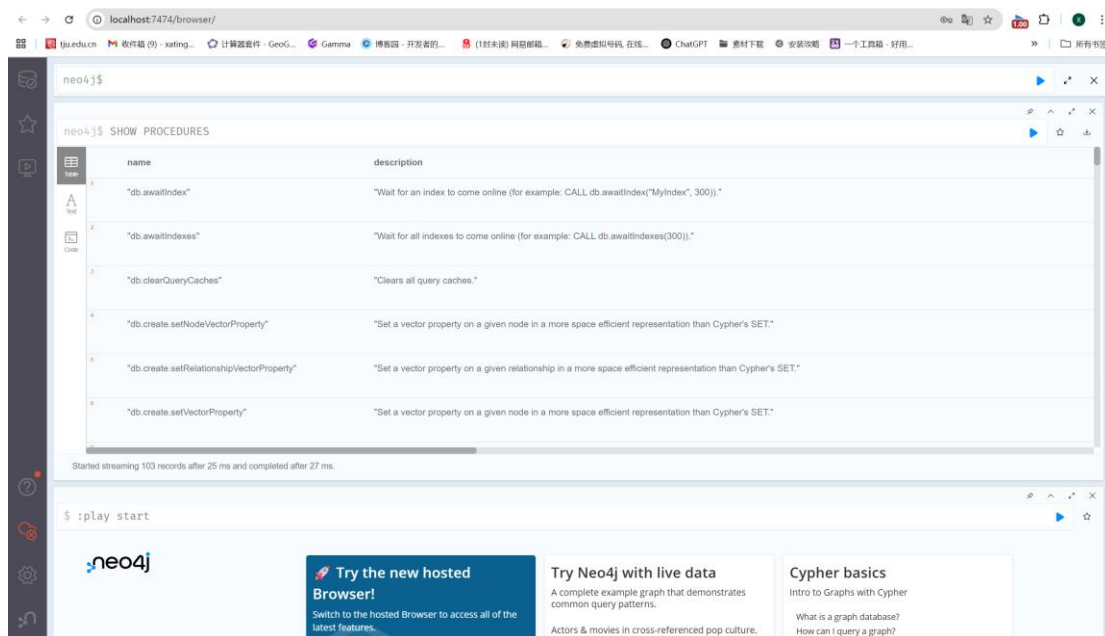
```
Windows PowerShell
安装最新的 PowerShell, 了解新功能和改进! https://aka.ms/PSWindows

加载个人及系统配置文件用了 5215 毫秒。
(base) PS E:\othersoftware\Neo4j\neo4j-community-5.26.12> .\bin\neo4j.bat start
Validating Neo4j configuration: E:\othersoftware\Neo4j\neo4j-community-5.26.12\conf\neo4j.conf
1 issue found.
Warning: Use of deprecated setting 'dbms.unmanaged_extension_classes'. It is replaced by 'server.unmanaged_extension_classes'.

Validating user Log4j configuration: E:\othersoftware\Neo4j\neo4j-community-5.26.12\conf\user-logs.xml
No issues found.

Validating server Log4j configuration: E:\othersoftware\Neo4j\neo4j-community-5.26.12\conf\server-logs.xml
No issues found.

Configuration file validation successful (with warnings).
Directories in use:
home:      E:\othersoftware\Neo4j\neo4j-community-5.26.12
config:    E:\othersoftware\Neo4j\neo4j-community-5.26.12\conf
logs:      E:\othersoftware\Neo4j\neo4j-community-5.26.12\logs
plugins:   E:\othersoftware\Neo4j\neo4j-community-5.26.12\plugins
import:    E:\othersoftware\Neo4j\neo4j-community-5.26.12\import
data:      E:\othersoftware\Neo4j\neo4j-community-5.26.12\data
certificates: E:\othersoftware\Neo4j\neo4j-community-5.26.12\certificates
licenses:  E:\othersoftware\Neo4j\neo4j-community-5.26.12\licenses
run:       E:\othersoftware\Neo4j\neo4j-community-5.26.12\run
Starting Neo4j.
Started neo4j. It is available at http://localhost:7474
There may be a short delay until the server is ready.
(base) PS E:\othersoftware\Neo4j\neo4j-community-5.26.12> |
```



运行

```
CALL n10s.graphconfig.init({
    handleVocabUris: 'MAP'
})
```

```
neo4j$ CALL n10s.graphconfig.init({ handleVocabUri: 'MAP' })
```

param	value
"handleVocabUri"	"MAP"
"handleMultival"	"OVERWRITE"
"handleRDFTypes"	"LABELS"
"keepLangTag"	false
"keepCustomDataTypes"	false
"applyNeo4jNaming"	false
"baseSchemaNamespace"	"neo4j://graph.schema#"
"baseSchemaPrefix"	"n4sch"
"classLabel"	"Class"
"subClassOfRel"	"SCO"
"dataTypePropertyLabel"	"Property"

Started streaming 17 records after 9 ms and completed after 241 ms.

运行

CREATE CONSTRAINT n10s_unique_uri FOR (r:Resource)
 REQUIRE r.uri IS UNIQUE;

```
neo4j$ CREATE CONSTRAINT n10s_unique_uri FOR (r:Resource) REQUIRE r.uri IS UNIQUE;
```

Added 1 constraint, completed after 151 ms.

Added 1 constraint, completed after 151 ms.

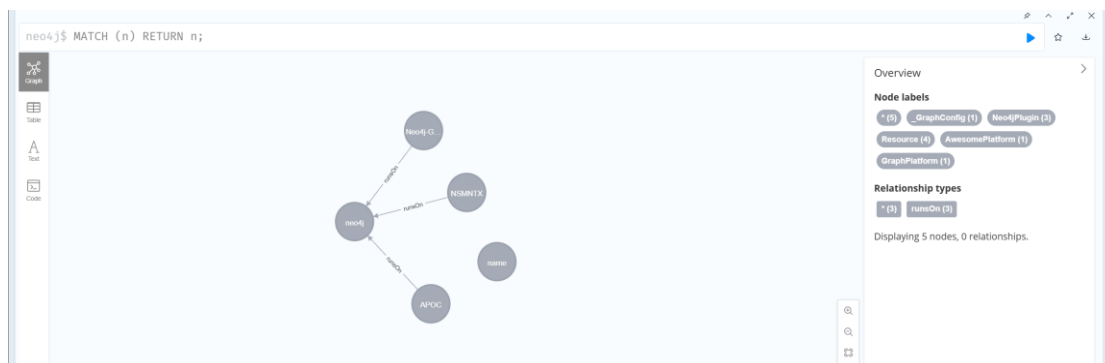
插入 RDF 数据

```
neo4j$ CALL n10s.rdf.import.inline( ' @prefix neo4voc: <http://neo4j.org/vocab/sw#> . @prefix neo4ind: <http://neo4j.org/ind#> . neo4ind:nsmntx...
```

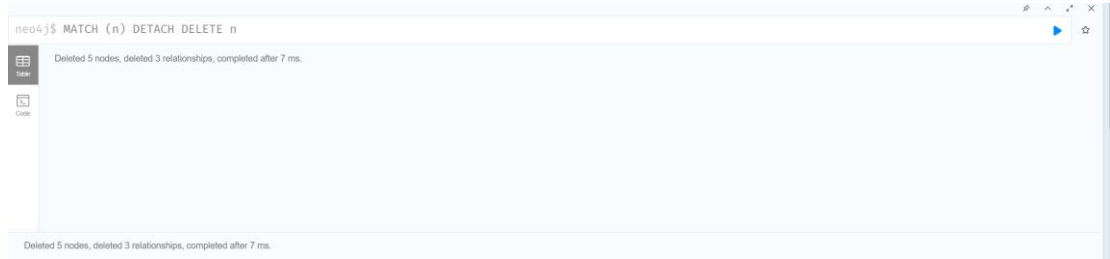
terminationStatus	triplesLoaded	triplesParsed	namespaces	extraInfo	callParams
"OK"	19	19	null	"	{ "singleTx": true }

Started streaming 1 records after 8 ms and completed after 238 ms.

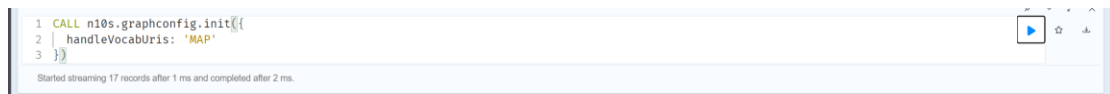
查询 RDF 数据



清空数据库



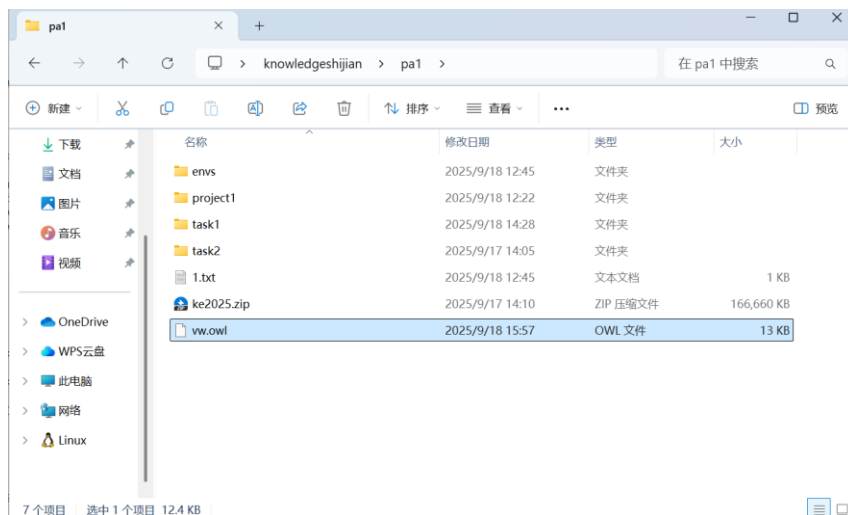
(2) 导入本体数据



导入本体数据



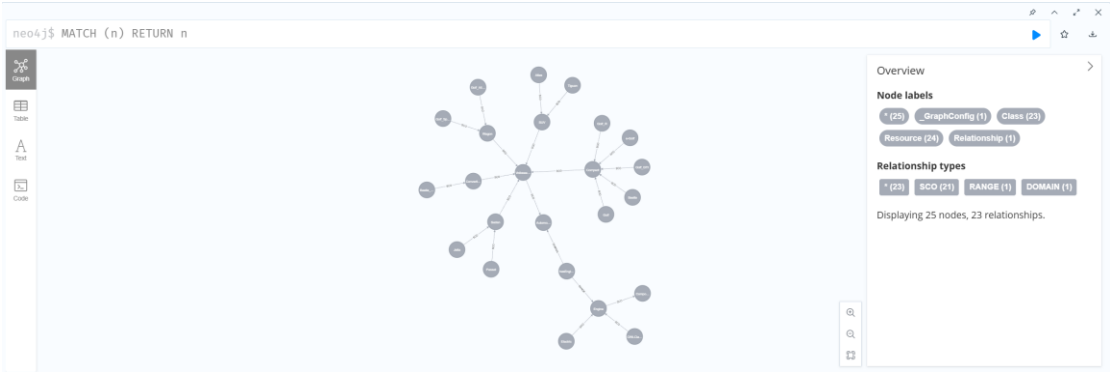
发现不能成功，于是下载文件



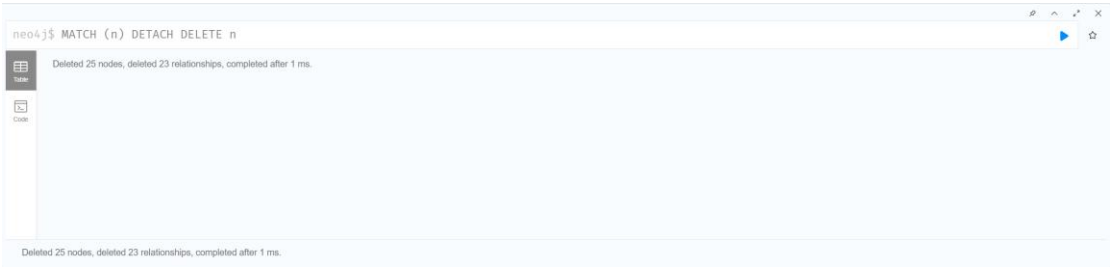
手动导入



发现成功导入
查询 MATCH (n) RETURN n 得到：

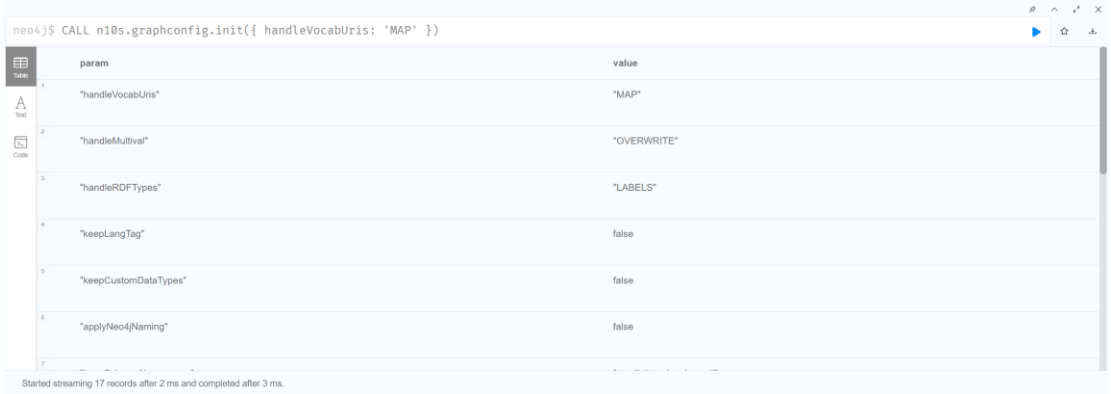


清空数据库



(3) RDF 数据类型推理

初始化插件

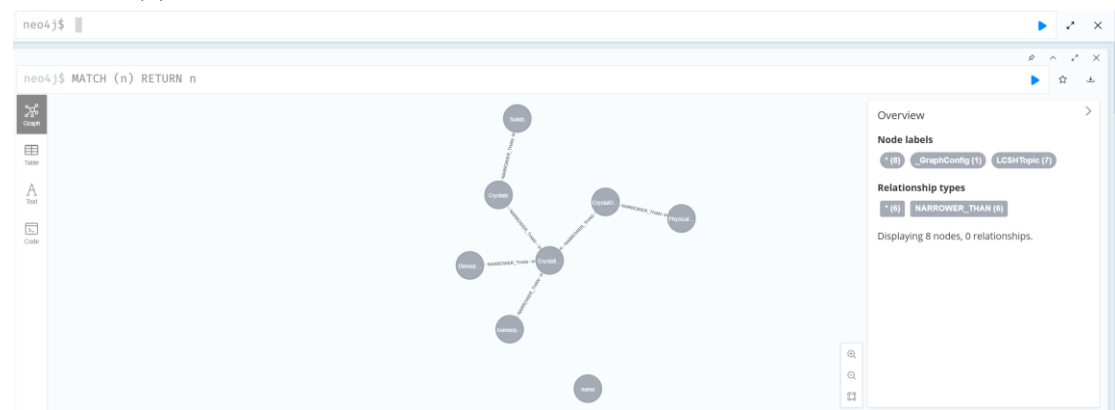


a) 建立 RDF 层次类型数据

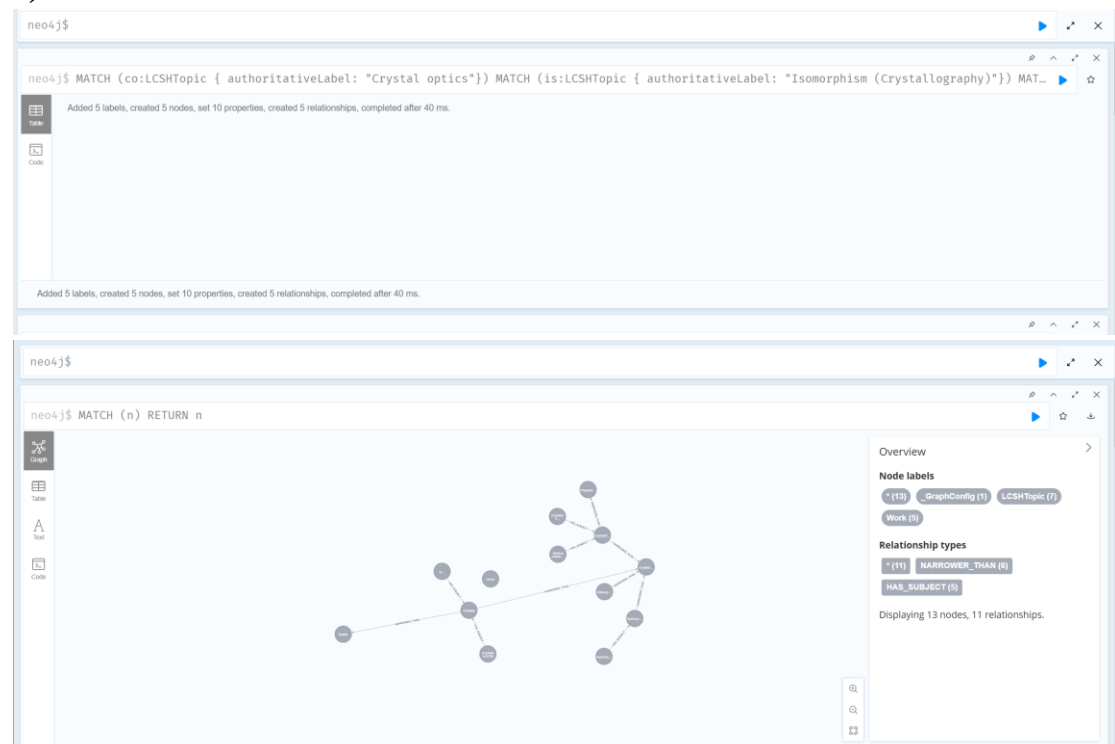
运行代码



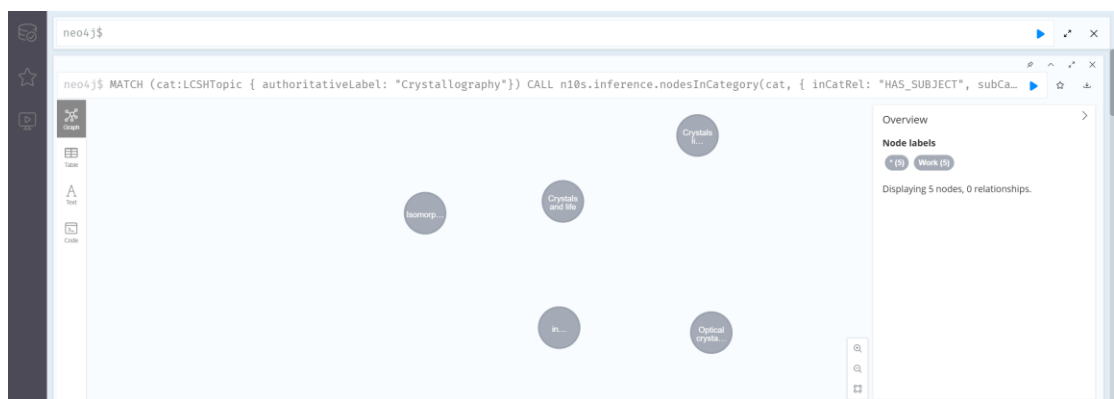
MATCH (n) RETURN n



b) 输入用于推理的数据



c) 运用推理进行类实例查询



(4) 运用推理判断类别实例

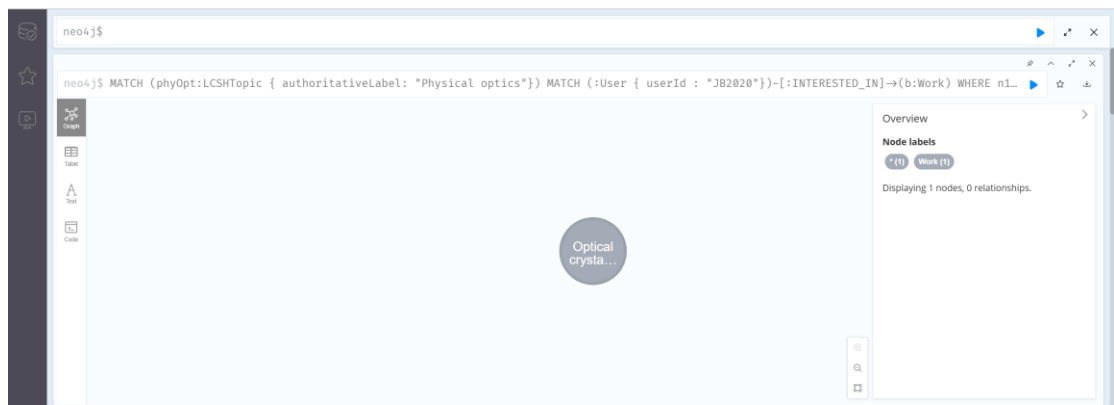
a) 建立数据



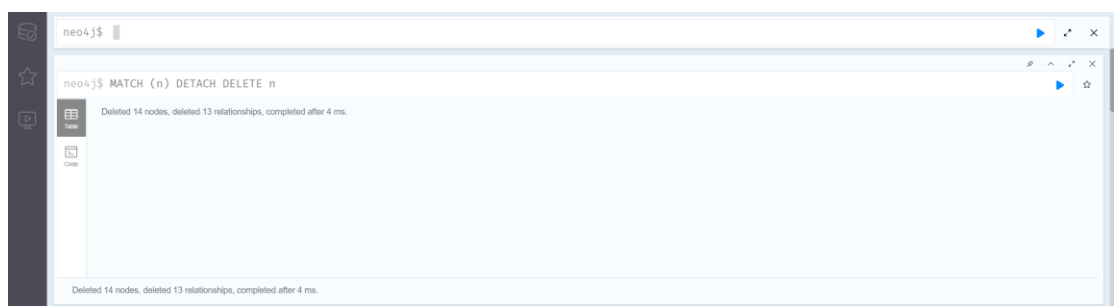
b) 建立参数变量



c) 进行查询



清空数据库



(5) 关系层次推理 初始化

```
neo4j$ CALL n10s.graphconfig.init({ handleVocabUri: 'MAP' })
```

param	value
1 "handleVocabUri"	"MAP"
2 "handleMultival"	"OVERWRITE"
3 "handleRDFTypes"	"LABELS"
4 "keepLangTag"	false
5 "keepCustomDataTypes"	false
6 "applyNeo4jNaming"	false
7	

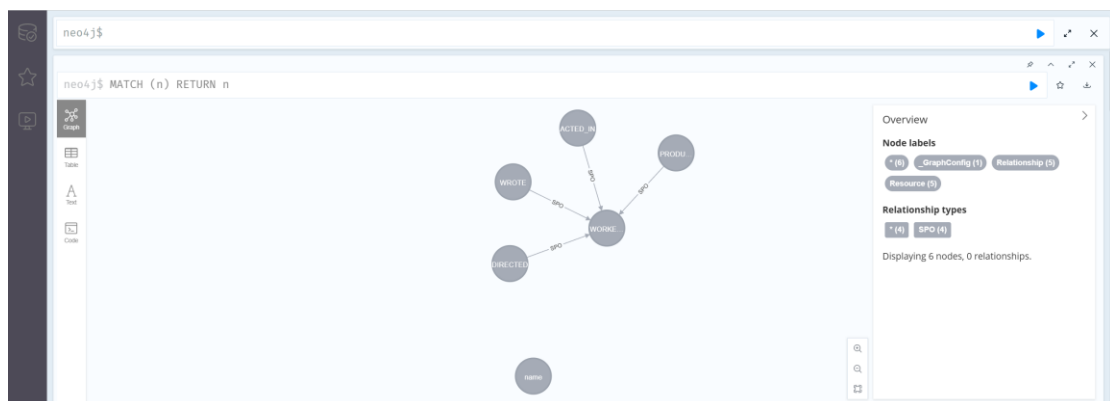
Started streaming 17 records after 3 ms and completed after 4 ms.

a) 建立数据

```
neo4j$ CALL n10s.onto.import.inline(' @prefix owl: <http://www.w3.org/2002/07/owl#> . @prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> . @_
```

terminationStatus	triplesLoaded	triplesParsed	namespaces	extrainfo	callParams
1 "OK"	14	14	null	-	{ "singleTx": true }

Started streaming 1 records in less than 1 ms and completed after 25 ms.



加载:movie 示例

```
neo4j$ $:play movies
```

Movie Graph

Pop-cultural connections between actors and movies

The Movie Graph is a mini graph application containing actors and directors that are related through the movies they've collaborated on.

This guide will show you how to:

- Create: insert movie data into the graph
- Find: retrieve individual movies and actors
- Query: discover related actors and directors
- Solve: the Bacon Path

WARNING: This guide will modify the data in the currently active database.

1/8 < ● ● ● ● ● ● >

neo4j\$

neo4j\$ match (thematrix:Movie {title: "The Matrix"}) call n10s.inference.getRels(thematrix,"WORKED_IN", { subRelRel: "SPO" }) yield rel, node r...

Graph

Table

Text

Code

Carrie A.

Laurence Fish.

Keanu Reeves

Hugo Weaving

Lilly Wach.

Lana Wach.

Emil Eifrem

Overview

Node labels

Person (7)

Displaying 7 nodes, 0 relationships.

neo4j\$

neo4j\$ match (thematrix:Movie {title: "The Matrix"}) call n10s.inference.getRels(thematrix,"WORKED_IN", { subRelRel: "SPO" }) yield rel, node r...

Graph

Table

Text

Code

relType	node
"DIRECTED"	(:Person {name: "Lana Wachowski",born: 1965})
"DIRECTED"	(:Person {name: "Lilly Wachowski",born: 1967})
"ACTED_IN"	(:Person {name: "Hugo Weaving",born: 1960})
"ACTED_IN"	(:Person {name: "Laurence Fishburne",born: 1961})
"ACTED_IN"	(:Person {name: "Carrie-Anne Moss",born: 1967})
"ACTED_IN"	(:Person {name: "Keanu Reeves",born: 1964})
"ACTED_IN"	(:Person {name: "Emil Eifrem",born: 1978})

MAX COLUMN WIDTH